

PROGRAMMING IN HASKELL

0



Chapter 4.3 - First Class Functions

Lambdas, let and where

1

We wish to write a function that takes two numbers, and returns the greater of the sum of the squares of the numbers $(x^2 + y^2)$ or the square of the sums of the numbers $(x+y)^2$

We will call it **sumSquareOrSquareSum** we will write this using :

- where
- lambda functions, and finally
- let

Starting point

Repetition of terms

2

```
sumSquareOrSquareSum :: Int -> Int -> Int
sumSquareOrSquareSum x y =
    if x^2 + y^2 > (x + y)^2
    then x^2 + y^2
    else (x + y)^2
```

We can do better....

Using where

3

```
sumSquareOrSquareSum' :: Int -> Int -> Int
sumSquareOrSquareSum' x y =
    if sumSquare > squareSum
    then sumSquare
    else squareSum
where    sumSquare = x^2 + y^2
        squareSum = (x + y)^2
```

- Cleaner
- Easier to read

Towards lambda functions

4

```
body :: Int -> Int -> Int
body sumSquare squareSum =
    if sumSquare > squareSum
    then sumSquare
    else squareSum
```

```
sumSquareOrSquareSum'' :: Int -> Int -> Int
sumSquareOrSquareSum'' x y =
    body (x^2 + y^2) ((x + y)^2)
```

Next, we will rewrite body as a lambda function.

Using lambda functions

5

Rewrite 'body as
lambda

```
sumSquareOrSquareSum''' :: Int -> Int -> Int
sumSquareOrSquareSum''' x y =
    (\sumSquare squareSum ->
     if sumSquare > squareSum
     then sumSquare
     else squareSum )
    (x^2 + y^2) ((x + y)^2)
```

Using let

Readability of the
where clause

6

```
sumSquareOrSquareSum''' :: Int -> Int -> Int
sumSquareOrSquareSum''' x y =
    let  sumSquare = (x^2 + y^2)
        squareSum = (x + y)^2
    in
        if sumSquare > squareSum
        then sumSquare
        else squareSum
```

With the power of
the lambda function

First Class Functions

7

First class functions are functions that can be passed around like any other values, including as arguments to functions.

First Class Functions. example

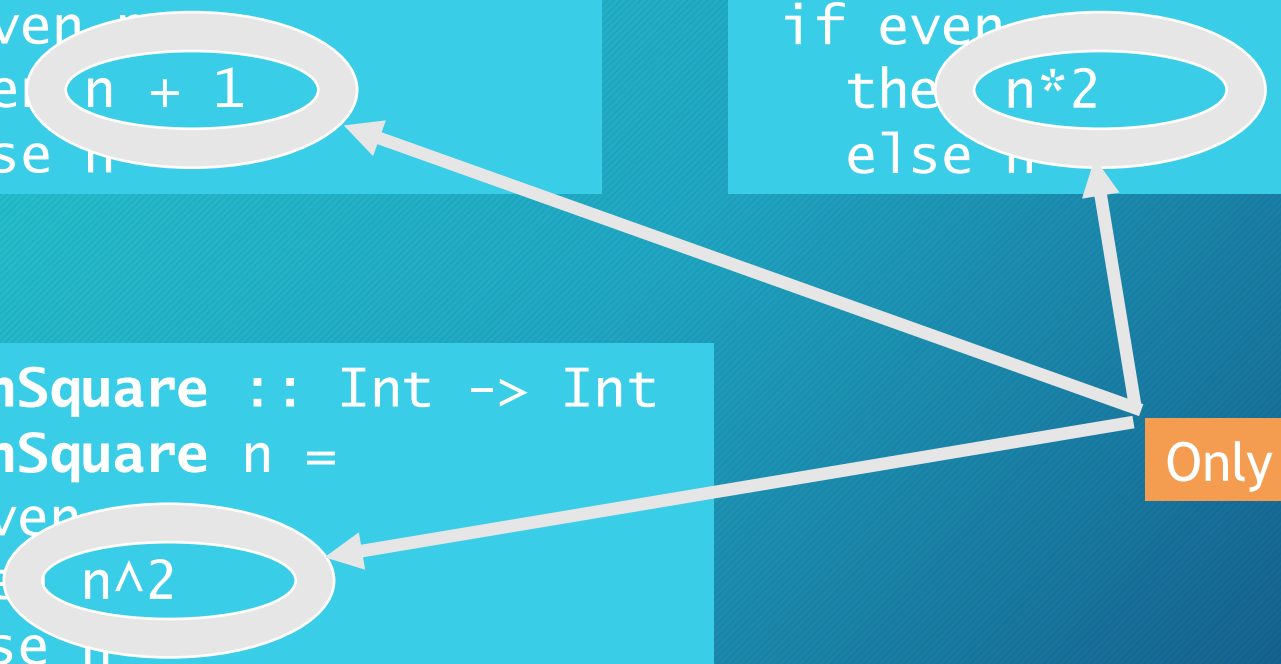
8

```
ifEvenInc :: Int -> Int
ifEvenInc n =
  if even n
  then n + 1
  else n
```

```
ifEvenDouble :: Int -> Int
ifEvenDouble n =
  if even n
  then n*2
  else n
```

```
ifEvenSquare :: Int -> Int
ifEvenSquare n =
  if even n
  then n^2
  else n
```

Only differences



First Class Functions - Introduce a function as an argument

9

```
inc :: Num a => a -> a
inc n = n + 1
```

```
double :: Num a => a -> a
double n = n * 2
```

```
square :: Num a => a -> a
square n = n ^ 2
```

```
ifEven :: Integral a => (a->a) -> a -> a
ifEven f n =
  if even n
  then f n
  else n
```

even only works
on integers

First Class Functions

10

We can call these functions thus:

```
*Main> ifEven square 4
16
*Main> ifEven inc 4
5
*Main> ifEven inc 5
5
*Main> ifEven double 4
8
*Main> ifEven double 5
5
*Main> ifEven square 4
16
*Main> ifEven square 5
5
```


Using lambdas

11

```
*Main> ifEven (\x -> x*2) 6  
12  
*Main> ifEven (\x -> x*2) 3  
3  
*Main> ifEven (\x -> x^2) 3  
3
```

etc.

The map Function

12

The higher-order library function called map applies a function to every element of a list.

```
map :: (a → b) → [a] → [b]
```

For example:

```
> map (+1) [1,3,5,7]  
[2,4,6,8]
```


The map Function

13

We will examine more higher-order functions later

```
map :: (a → b) → [a] → [b]
```

For example:

```
> map toUpper "cat"  
> "CAT"
```


Example - sorting 1/3

14

In this sorting example we use:

- built in functions and also
- pass functions into other (sortBy) functions

We use:

- Pairs/Tuples (fst and snd return the first and second element of the tuples respectively)
- Form ("firstname", "lastname")

Example - sorting 2/3

15

```
names :: [(String, String)]
names = [ ("Hugo", "Keenan"),
          ("Mack", "Hansen"),
          ("James", "Lowe"),
          ("Bundee", "Aki"),
          ("Robbie", "Henshaw"),
          ("Sam", "Prendergast"),
          ("Dan", "Sheehan"),
          ("Jack ", "Crowley"),
          ("Caelan", "Doris"). ]
```

Default
sort is on
first
element

Called as

```
ghci> sort names
[("Bundee","Aki"),("Caelan","Doris"),("Dan","Sheehan"),("Hugo","Keenan"),("Jack ","Crowley"),("James","Lowe"),("Mack","Hansen"),("Robbie","Henshaw"),("Sam","Prendergast")]
```


Example - sorting 3/3

```
import Data.List ( sort, sortBy )

names :: [(String, String)]
names = "as before"

compareLastNames :: Ord a => (a, a) -> (a, a) -> Ordering
compareLastNames name1 name2
  | lastName1 > lastName2 = GT
  | lastName1 < lastName2 = LT
  | otherwise = EQ
  where lastName1 = snd name1
        lastName2 = snd name2
```

Called as

```
ghci> sortBy compareLastNames names
[("Bundee","Aki"),("Jack ","Crowley"),("Caelan","Doris"),("Mack","Hansen"),("Robbie","Henshaw")
,("Hugo","Keenan"),("James","Lowe"),("Sam","Prendergast"),("Dan","Sheehan")]
ghci> □
```